Unit 3. Data Preprocessing

Machine Learning 1 (ML1)

Master in Artificial Intelligence (MIA)

November 13, 2025

3.1 Introduction

We have a **learning algorithm** and a **dataset** to solve a problem...

- The **dataset** is formed by tuples (x, y) called **instances** or **examples**, where x is the **input**, expressed as a vector of **features**, also called **attributes** or **variables**, and y is the **expected output**.
- We want a **model** or learner that, in presence of new data, is capable of making **predictions** consistent with the initial dataset.

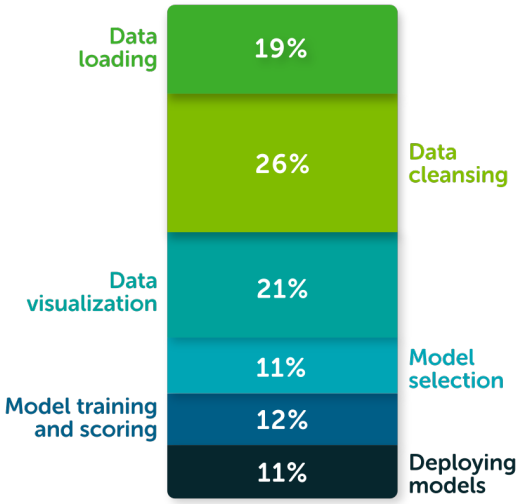
We will have to develop an **experimental design** in three main steps:

1. Data **preprocessing**.
2. **Training** of the model.
3. Estimation of the real **error** of the model.

Of these, we may expect the data preprocessing to be the simplest: just collect the data you need and feed it to your algorithm to train a model.

Then we find survey questions like this one:

"*How much of your time is spent on the following tasks*"



Source: The State of Data Science 2020 - Anaconda

Why? **Real world data is imperfect** and that has to be dealt with:

- Extracted data can be **noisy**, and have parts that are **lost** or be **biased**, leading to the extraction of useless patterns.
- Preprocessing can be used to **reduce** the size of the **original dataset**, improving the efficiency of subsequent learning.
- Machine learning **models** are **only as good as the data** that is used to train them (garbage in, garbage out).

So an **adequate preparation** of the data is important to ensure the **quality** in training and in the resulting models.

In this unit, we are going to focus on a couple of tasks involved in data preprocessing:

1. Data transformation
2. Data reduction

3.2 Data Transformation

Data transformation is the process of changing the **format**, **structure**, or **values** of **data** for **model building** and **data discovery** in general.

It is necessary because:

- Even after the compilation of a **dataset** with real world data there may be still **problems** in it (outliers, missing data, noisy data, ...).
- The definition of the **features** compiled may not be the best for the task at hand.
  - Some learning algorithms tend to pay more attention to features with **broad ranges**, so we may want to **scale** or **normalize** them.
  - Some attributes may be defined as **continuous values**, while they could be more useful in a **discrete representation**.
  - We may have some **fine-grained** values that would be more useful for the task we want to solve if **aggregated** or **generalized** in a single feature.

    ...

We will devote some space to some of the **data transformations** commonly preformed to generate useful datasets.

1. **Data cleaning**
2. **Feature scaling**
3. **Aggregation**
4. **Feature creation**
5. **Generalization**
6. **Discretization**

3.2.1 Data cleaning

Data cleaning is the process of **preparing data** for analysis by removing or modifying data that is **incorrect**, **incomplete**, **irrelevant**, **duplicated**, or **improperly formatted**.

Some of the tasks involved in this process are:

- **Erasing** or **inferring** missing data.
- **Smoothing** out **noisy data**.
- **Deleting** out of range data.
- **Correcting** inconsistencies.

We will focus on **the first two**.

Missing data

Missing data occurs when there is **no data** stored for a **variable** of interest in a **dataset**.

Depending on its volume, missing data can **damage** the **inference** of data analysis and the **robustness** of a machine learning model:

- **Loss of efficiency**.
- **More complexity** in data transformation and analysis.
- **Biases** caused by **differences** between the complete dataset and the dataset with **null** (missing) **values**.

Before deciding what to do with missing values it is necessary to determine the **reason of their absence**.

- Are the null values **really significant** to the training of our model? Can they be ignored?

One possibility is that null data exists is not due to error or bias, but because the **absence of data has some meaning**.

E.g., "*If temperature exceeds X °C, it is necessary to know the pressure*".

- In this case, the dataset will have attributes for temperature and pressure.
- In those instances in which temperature is lower than X °C, the variable pressure will have **null data** ⇒ pressure is **conditional data**.
- We may have to find ways to work conditional data into our models.

On the other hand, when data is extracted from **surveys**, human **peculiarities** and **errors** can produce three kinds of **lost data**.

- **Missing completely at random** (MCAR): loss of data has **nothing to do with the person** being studied.

  E.g., a questionnaire might be lost in the mail, or a blood sample might be damaged in the lab, ...
- **Missing at Random** (MAR): loss of data has **nothing to do with the person** being studied, and **can be predicted** from other information about them.

  E.g., men and women have different response rates when asked about their age. Not answering depends on other variable (gender).
- **Missing not at random** (MNAR): missingness of data is **specifically related** to the data in question.

  E.g., a person missing a drug test because they took drugs the night before.

MCAR and **MAR** can be solved by **imputation**. In the case of **MAR**, we can try to **infer** missing values from **other features**.

MNAR data is **more problematic**, because:

- If treated like MAR/MCAR, we are introducing **bias**.
- In many cases it is **not possible** to know if null values are MCAR, MAR or MNAR.
- Many methods to deal with missing data generally start from **MAR or MCAR assumptions**.

Now we will present some of the **most common ways** of dealing with missing data in machine learning.

A) The most obvious one is to let the **learning algorithm deal** with the null values.

- Some **learning algorithms can work** with missing values (e.g., some implementations of C4.5) or **learn** by themselves **values** for them on training loss reduction (e.g., XGBoost).
- **Pros**: we do not have to do anything!
- **Cons**: **most** learning algorithms **do not** work with missing data.

B) Another possibility is to **fill the data manually**.

- If some **answers** from a survey are **missing** just go back and **repeat** it.
- Have a **human expert** fill the **missing data**.
- **Pros**: probably the only option that best guarantees **data quality**.
- **Cons**: very **costly**, unfeasible in all but the **smallest datasets**.

C) A more used solution is **deletion**, with two possibilities.

1. **List-wise deletion**: **instances** with missing data are **deleted**.
   - One of the **most used** solutions.
   - **Pros**:
     - **Simple** to implement.
     - Resulting models still work well if null data is **MCAR**.
   - **Cons**:
     - It **discards valid data**, which could be useful in training.
       If many features or instances are affected by null data, a lot of **valuable data** will be **lost**.
     - If null data is **MAR** or **MNAR**, the resulting dataset could be **biased**.
2. **Discard the feature**.
   - **Not recommended**.
   - Feasible only if:
     - there are **many null values** (the feature is useless), or
     - **MAR assumption holds**: there is another attribute for a dependent variable with better data.

D) Finally, we can **replace** the missing data with **inferred values**, in a process called **imputation**. Some currently used methods are:

1. Impute with the **average/median/mode** of the values present in the dataset for this atribute.
   - It can be done with **all instances** or only with those in the **same class** that the one with missing data.
   - **Pros**: **easy** and **fast**, **escales well** for large datasets.
   - **Cons**:
     - Does not factor **correlations** between features.
     - May introduce **bias** in the data.
     - **Poor** results on **categorical features**.
2. Imputation with **most frequent** or **zero/constant** values.
   - **Pros**: better than average/median/mode for **categorical features**.
   - **Cons**:
     - Does not factor **correlations** between features.
     - May introduce **bias** in the data.
3. **Hot deck imputation**: missing value is imputed from a **randomly selected** or **similar** example, called **donor**. In the latter case, similarity is computed to some **metric** based on the values of its attributes.
   - **Pros**:
     - Tends to avoid **inconsistencies** better than previous methods.
     - Works well with **MCAR** data.
   - **Cons**: performance **depends** on the existance of **suitable donors**.
4. Imputation using the **Expectation-Maximization** (EM) algorithm.
   - Well known **iterative method** to find **maximum likelihood estimates** of parameters in statistical models, where the model depends on unobserved latent variables.
   - **Pros**: **easy** to implement, applicable in most cases, and **effective**.
   - **Cons**: it does not produce **standard errors** of estimates.
5. **Imputation using a model**: a **model** is trained to **predict** the missing values of an attribute using the **other features** of instances without null data.

E.g., we try to estimate a model for the missing x_{23} using the rest of the features, and instances x_1, x_3, x_4, and x_5.

Initial data:

x_{11}	x_{12}	x_{13}	x_{14}	y_1
x_{21}	x_{22}	NULL	x_{24}	y_2
x_{31}	x_{32}	x_{33}	x_{34}	y_3
x_{41}	x_{42}	x_{43}	x_{44}	y_4
x_{51}	x_{52}	x_{53}	x_{54}	y_5

$\Rightarrow$

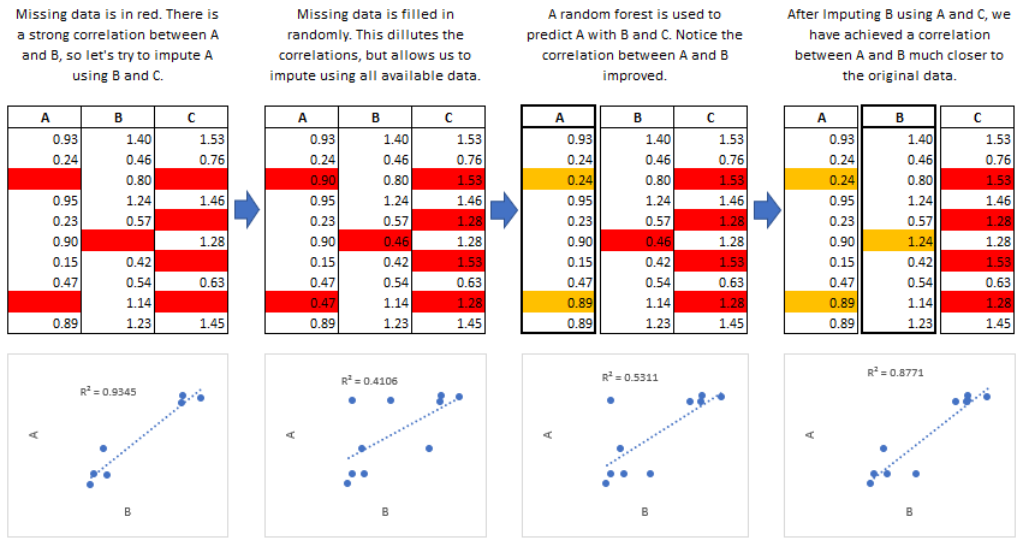
Data for imputation:

x_{11}	x_{12}	x_{14}	y_1	x_{13}
x_{31}	x_{32}	x_{34}	y_3	x_{33}
x_{41}	x_{42}	x_{44}	y_4	x_{43}
x_{51}	x_{52}	x_{54}	y_5	x_{53}

- With **k-NNs** the estimated value could be the **average** of the k nearest neighbors.
- We can introduce the output y as an **attribute** in the new model if the original problem is a **regression**.
- Otherwise, we **discard** y, and we can train a **single model** for the whole dataset, or **one model per class** in y.
- **Pros**: can be much **more accurate** than the mean, median or most frequent imputation methods (it depends on the dataset).
- **Cons**: more **time intensive** than previous methods. We have to be careful and taylor the **complexity** of the new model(s) to the data.

6. **Multiple imputation**.
   - Form of **stochastic** imputation, filling **all missing values** of a dataset (not limited to a single feature), by generating m **possible imputations** for each missing value (and thus m possible datasets), and them **combining** them.
   - Popular models: Multiple Imputation by Chained Equations (MICE), Multiple Imputation with Denoising Autoencoders (MIDAS).
   - **Pros**:
     - Works well with **MCAR** and **MAR** data.
     - Can **preserve correlation** between features.
     - Takes into account the **uncertainty** in the imputations.
   - **Cons**: more **complex** and computationally **costly**.

MICE is one of the most popular methods for multiple imputation today. Let's take a look at how it works.



Source: The MICE algorithm, by Sam Wilson

The following procedure is repeated m times:

1. Calculate a **placeholder** (e.g., random value, imputation by mean, ...) for every **missing value** in the dataset.
2. Set back to **missing** the placeholder imputations for **one of the features**.
3. Build a **regression model** to impute the missing values in that feature with the rest of the attributes, dropping the instances with missing values.
4. Use the regression model to **predict the missing values**.
5. **Repeat** steps 2-4 for **every feature** with missing values until imputing all of them.
6. **Repeat** steps 2-5 in several cycles, **updating** in each cycle the imputed values of the previous one. A complete dataset is obtained at the end.

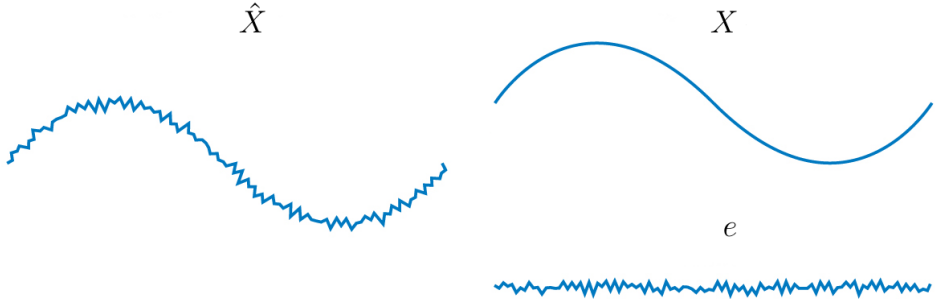
We can use the final dataset or the m (imputed) datasets to train models.

Noisy data

Traditionally, noisy data is modeled as a **random error** in the variables being measured.

Given a **variable** $\boldsymbol{X}$, its **observed values** $\hat{\boldsymbol{X}}$ are a sum of its **real values** and **noise**:

$$\hat{X} = X + e$$



We want to **smooth** the observed values of $\hat{\boldsymbol{X}}$ so $\boldsymbol{e}$ is **minimized**. Two of the most basic methods are **binning** and **linear regression**, but **outlier detection** can also be used.

A) In **Binning**, a series of **ranked values** are grouped in consecutive intervals, or **bins**, and the values in each interval are smoothed separately.

There are 2 basic methods of dividing data into bins:

1. **Equal Frequency Binning**: bins have an equal frequency (same number of values).
2. **Equal Width Binning**: bins have the same width defined as:

$$w = \frac{\max - \min}{\#bins}$$

Once the data is conveniently divided, we can apply very simple smoothing techniques:

- **Smoothing by bin means/medians**: replace the values in the bin by its mean or median.
- **Smoothing by bin boundary**: the minimum and maximum values in a given bin are identified as the bin boundaries. Each bin value is replaced by the **closest boundary** value.

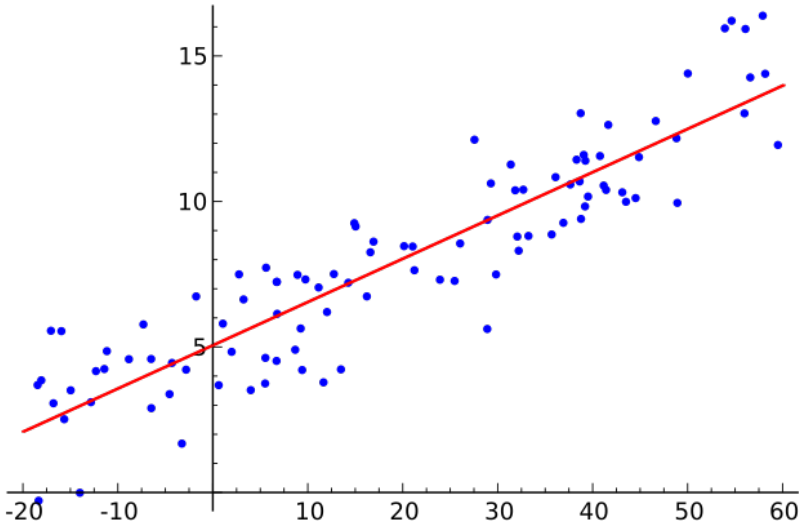
Example: we can divide the ranking $4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34$ in three bins and smooth their values.

		Smoothing by bin...		
Equal freq. binning		means	medians	boundaries
Bin 1	$4, 8, 9, 15$	$9, 9, 9, 9$	$9, 9, 9, 9$	$4, 4, 4, 15$
Bin 2	$21, 21, 24, 25$	$23, 23, 23, 23$	$23, 23, 23, 23$	$21, 21, 25, 25$
Bin 3	$26, 28, 29, 34$	$29, 29, 29, 29$	$29, 29, 29, 29$	$26, 26, 26, 34$
Equal width binning		means	medians	boundaries
4-14	$4, 8, 9$	$7, 7, 7$	$8, 8, 8$	$4, 9, 9$
15-24	$15, 21, 21, 24$	$20, 20, 20, 20$	$21, 21, 21$	$15, 24, 24, 24$
25-34	$25, 26, 28, 29, 34$	$28, 28, 28, 28, 28$	$28, 28, 28, 28, 28$	$25, 25, 25, 25, 34$

(Note: we use integer values, and the median of an even number of observations is the average of the two middle ones)

B) Another method used for smoothing noisy data is **Linear Regression**:

- Given a set of tuples representing the corresponding values of **two variables**, linear regression finds the **linear function** that best fit the data.



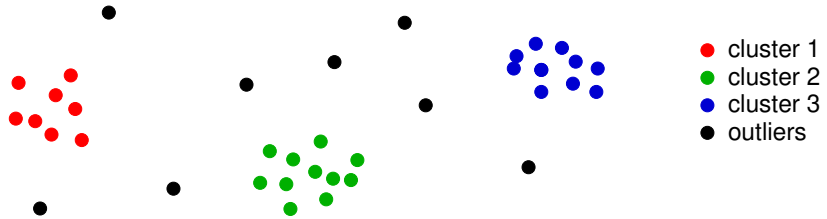
Source: Wikipedia (Author: Sewaqu)

- Linear regression models are often fitted using the **least squares approach**, minimizing the **sum of mean squared loss** between observed and predicted values.
- If the variables involved are **more than two**, we have a **multiple linear regression**.
- The **goal** is to find a mathematical expression expressing the **underlying relationship** between variables and use it to smooth noisy observations.

  Noisy values are replaced by the values **predicted** by the linear function.

C) Noise reduction can also be accomplished by **outlier detection**.

- **Outliers** or anomalies are defined as objects with behaviors very **different from expectation**.
- They can be detected in an unsupervised manner via **clustering** techniques.
  - Clustering is the process of **grouping objects** based on the principle of maximizing intraclass similarity and minimizing interclass similarity.
  - Clustering allows us to **group instances** in the input data. Examples **outside** clusters will be considered **outliers**.



- Another possibility is **proximity-based methods**:
  - As in clustering, **distance** between vectors is used to quantify the **similarity** between instances.
  - Instances that are **far from other** instances can be regarded as **outliers**.
  - A **distance-based** outlier detection method consults the **neighborhood** of an example, defined by a given **radius**.
  - The example is then considered an **outlier** if its neighborhood does not have enough other **instances**.
- Independently of the technique used, **elimination of outliers** will contribute to **noise reduction** in the dataset.

3.2.2 Feature scaling (data normalization)

Preprocessed data may contain attributes with **different scales** for **various quantities** such as money, weights, distances, ...

This is a significant problem as more than a few machine learning **algorithms** are **highly sensitive** to the **ranges** of these features.

- In general, learners that use **gradient descent** as an optimization technique require data to be scaled, since the **input** is directly involved in the **computation of gradients**.

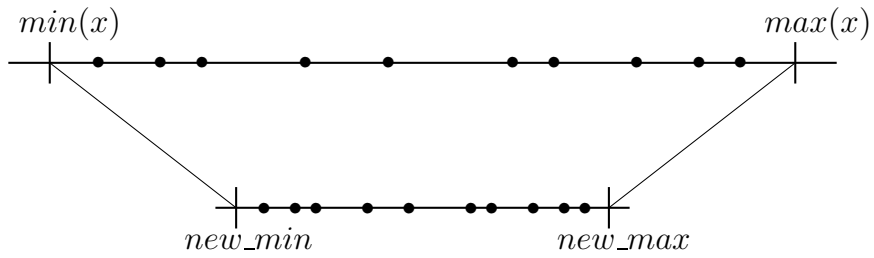
  They include, but are not limited to, are Linear Regression, Logistic Regression, ANNs, ...
- **Distance-based algorithms** like k-NN, k-means, and SVMs are among the **most affected** by the range of features, because distance between data points is how they usually determine their similarity.
- The **exception** are **Decision Trees**, and other tree-based learning algorithms, **Naive Bayes**, or **Linear Discriminant Analysis**.

  DTs split nodes based on a single feature each time, so differences in range between attributes are not an issue.

As a result, it is important to make sure that the range of the different attributes is our data is scaled. This task is commonly known as **feature scaling** or **data normalization**.

The following are the main scaling methods of continuous variables:

1. **Normalization (min-max normalization, rescaling)** is the simplest method, and consists in rescaling the range of a feature $[min, max]$ to a new interval $[new_min, new_max]$.



$$v' = \frac{v - min}{max - min} \times (new_max - new_min) + new_min$$

where v is the original value and v' the normalized one.

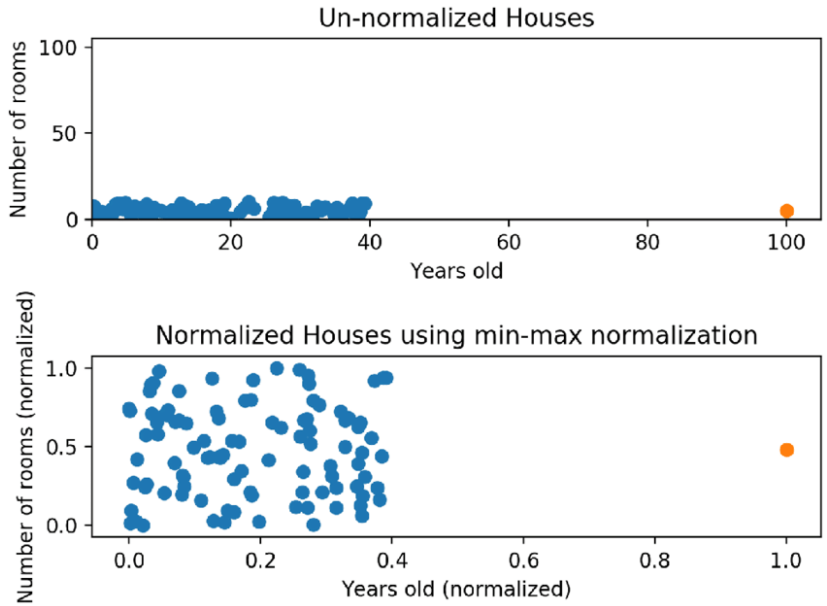
The most usual case is when the new **range** is $[\mathbf{0}, \mathbf{1}]$, which simplifies the equation to:

$$v' = \frac{v - min}{max - min}$$

Another popular range is $[-\mathbf{1}, \mathbf{1}]$.

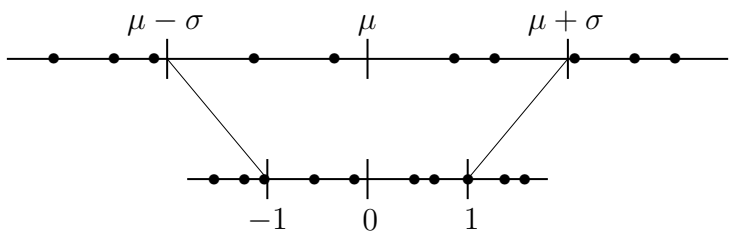
This type of normalization just changes the **scale** of the data:

- It preserves the **order** and **relative distances** between values.
- It works well when the **data** is **bounded** by the **maximum** and **minimum** of the original interval, and there are **no outliers**.
- If there is one or more **outliers**, they will set the maximum and/or minimum of the original interval, scaling non-outlier values to a very **small range**, where they can be mistaken as **noise**.



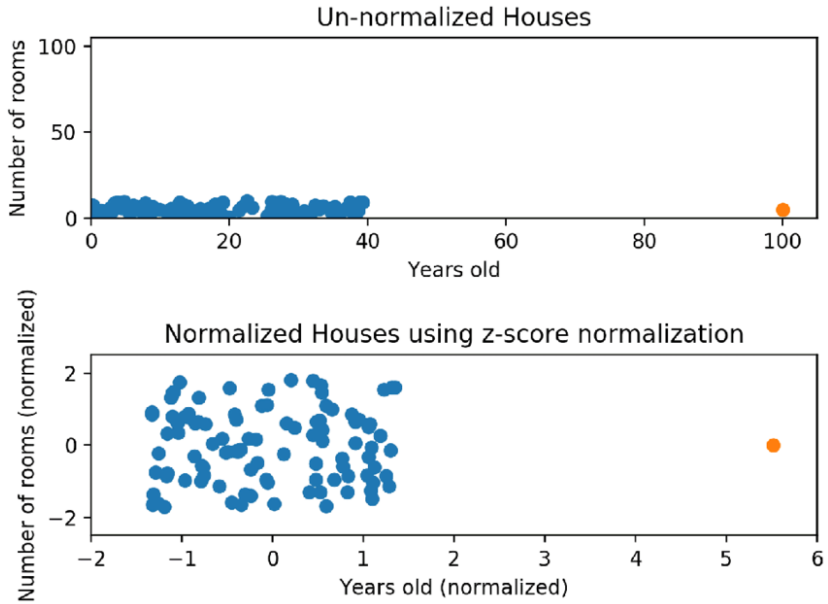
Source: J. Rudd, H. Ray. An Empirical Study of Downstream Analysis Effects of Model Pre-Processing Choices. (link)

2. **Standarization (Z-score normalization)**: attribute is changed to have **zero mean** ($\mu = 0$) and **unit standard deviation** ($\sigma = 1$).



$$v' = \frac{v - \mu}{\sigma}$$

where $\boldsymbol{\mu}$ is the **mean** of the original data and $\boldsymbol{\sigma}$ its **standard deviation**. Since the values are not **restricted** to a particular range, this kind of scaling is more **robust to outliers** than min-max normalization.



Source: J. Rudd, H. Ray. An Empirical Study of Downstream Analysis Effects of Model Pre-Processing Choices. (link)

Normalization and standarization are the most popular scaling techniques. It is interesting to contrast them:

Normalization	Standarization
Scales values to a new range, usually $[0, 1]$	Centers data around the mean and scales to a std. dev. of 1. Not bounded by a range
Useful when data distribution not Gaussian or unknown	Works better with a Gaussian distribution
Sensitive to outliers	Less sensitive to outliers
Retains shape of the original data distribution	Retains shape of the original data distribution

3. **Decimal scaling**: the **decimal point** is **moved** to the **left** in all values of the attribute, i.e. divided by a power of 10, until all of them are in the interval $(-1, 1)$.

$$v' = \frac{v}{10^j}$$

where j is the smallest integer such as $\max(|v'|) < 1$

E.g., 15.3, 8.34 and 732.83 will be scaled to 0.0153, 0.00834, 0.73283, with $j = 3$.

4. **Scaling to unit length**: all components of the feature vector are scaled so that it has length one. It is usually done by dividing each component by the **euclidean length** of the vector.

$$x' = \frac{x}{\|x\|} \quad \text{where} \quad \|x\| = \sqrt{x \cdot x} = \sqrt{\sum_{i=1}^{n} x_i^2}$$

Unlike the previous kinds of scaling, that can be used separately on a given attribute, scaling to unit length is **applied over all features** of the instance.

It is important to remember that whichever **scaling** have been used in the **training dataset** must be **preserved** for future use during the operation of the system on new data, including its **parameters** ($min, max, \sigma, \mu, \ldots$).

3.2.3 Aggregation, feature construction, generalization and discretization

Data cleaning and normalization involved **adding** or **changing** values to existing **features**.

Now we are going to take a look at some operations that create **new features** from existing ones.

- **Goal**: reducing **complexity**.
- **Means**: by giving the training algorithm **attributes** more **tailored** to the task we want to see accomplished.

A) In **aggregation**, summary or aggregation operations are applied to the features.

- A **new attribute** is created that represents the **sum** (or a similar operation) **over the values** of other attributes or instances.
  - For **numeric data**, typical operations are: sum, substraction, product, division, maximum, minimum, average, ...
  - For **categorical data**, usually logic operations are used: conjunction (and), disjunction (or), negation, implication, ...
- It can be interpreted as **moving up** in a **hierarchy** of numeric attributes.

  E.g., the daily sales data may be aggregated into monthly and annual total amounts $\Rightarrow$ having only daily data adds unneeded complexity.

B) In **feature construction** new attributes are constructed and added to the given set of attributes to help the processing of the learning algorithm.

- If we can **combine** several **features** with an expression that can be useful, we can spare the algorithm from having to learn it by itself.
  - It should be avoided if the learning algorithm is capable of inferring that expression.
  - Domain knowledge determines which new attributes should be built.
- In **numeric attributes** usually involves applying a **numerical formula**.

  E.g., body_mass_index = height_in_cm^2/weight_in_Kg.

- In **categorical atributes**, **logical operations** and **predicates** are usually involved.

  E.g., male$(x) \wedge$ married$(x) \wedge$ children$(x) > 0 \rightarrow$ family_man(x).

  E.g., 3-of-5(fever, cough, tiredness, loss of taste/smell) $\rightarrow$ COVID-19.
- We also can generate a **numeric attribute** from a **combination** of numerical and categorical ones.

  E.g., X-of-N(age < 25, male, years since driving license < 2, sports car) $\rightarrow$ car insurance risk = X (the number of arguments that are true).

C) Generalization is the process of replacing categorical values by more abstract ones, based on **concept hierarchies**.

- It can be interpreted as **moving up** in a **hierarchy** of nominal attributes.

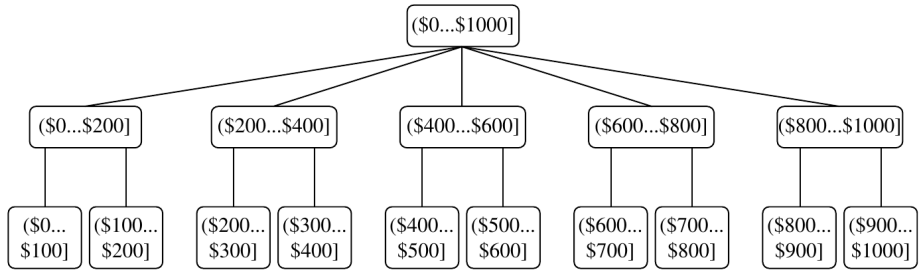
  E.g., Location hierarchy: country $\Rightarrow$ region $\Rightarrow$ city $\Rightarrow$ street.

  We can replace street by city if only the latter information is useful $\Rightarrow$ having other elements of the location adds unnecessary complexity.

D) In **discretization** the raw values of a **numeric** attribute are replaced by **interval labels** or **conceptual labels**.

  E.g., Age in years replaced by ten year intervals $(0\text{-}10, 11\text{-}20, \ldots)$ or categories like "youth", "adult", "senior".
- The labels, in turn, can be recursively organized into higher level concepts, resulting in a **concept hierarchy** for the numeric attribute.

  E.g., A concept hierarchy for the attribute price.



Source: Data Mining, Concepts and Techniques, pp. 112

- Necessary for building association **rules** and **classifiers** that only accept **discrete data**.

Discretization techniques can be **categorized** based in several criteria. The two most relevant for us are:

- **Supervised vs. unsupervised**: class information is used (or not).
- **Top-down vs. bottom-up**:
  - The former starts from an **empty list** of split points (i.e. all data in a single interval) and **adds more** in a recursive manner.
  - The latter starts from **all possible** split points (i.e. each value in a separate interval) and recursively **erases** them.

There are many potential discretization techniques:

- **Binning**: **top-down** and **unsupervised**.
- **Clustering**: **top-down/bottom-up** and **unsupervised**.
  - Each cluster is a node in the concept hierarchy.
- **Entropy-based**: **top-down** and **supervised**.
  - Similar to how **decision** trees are built.
  - Select **split-points** so that the resulting partition contains as many tuples of the same class as possible.
  - Usually the value with **minimum entropy** is selected as a split-point.
- **Based on correlation measures**: **bottom-up** and **supervised**.
  - For accurate discretization, the relative **class frequencies** should be fairly **consistent** within an interval.
  - If two **adjacent intervals** have a very **similar distribution** of classes, based on some statistical test, then the intervals can be **merged**.
    E.g., *ChiMerge*, a discretization method based on the χ^2 (chi-squared) test.

...

3.3 Data Reduction

Data reduction is the generic name of a series of techniques to obtain a **reduced representation** of a **dataset** much smaller in volume, yet closely maintaining the **integrity** of the original data.

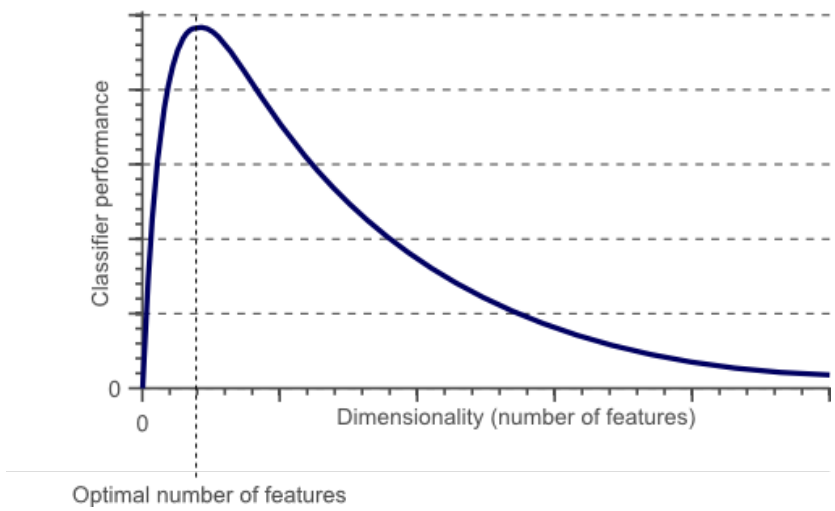
There are **two basic ways** of accomplishing data reduction:

1. **Numerosity reduction**: the **data volume** is **decreased** by selecting an alternative, smaller form of data representation. Several techniques:
   - **Regression** and **log-linear models**: data in a high-dimensional space is represented by models in lower dimensional spaces.
   - **Clustering**: instances ara grouped in clusters and representations of the clusters replace actual data.
   - **Sampling**: large data is represented by a much smaller random data sample (a subset of instances).
   - **Aggregation**, **discretization**, **generalization** of attributes.
2. **Dimensionality reduction**: transformation of data from a **high-dimensional space** into a **low-dimensional space**.
   - The low-dimensional representation **retains** some meaningful **properties** of the **original data**.
   - An attempt to deal with the "curse of dimensionality".

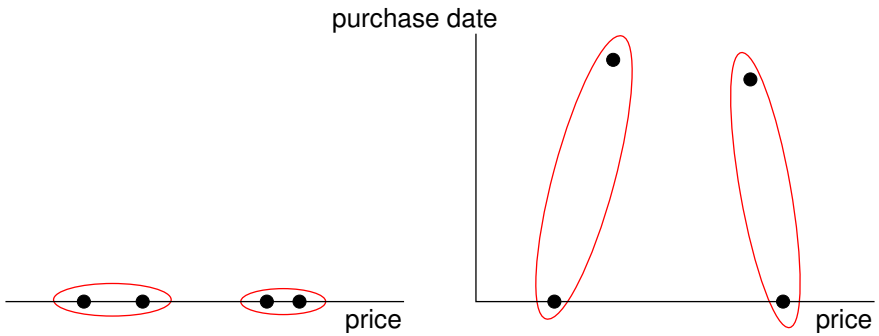
3.3.1 Dimensionality reduction

The **curse of dimensionality** is a mayor problem in data mining and ML.

1. The **number of examples** necessary to train a model with adequate generalization **grows exponentially** with the number of features.
   - **Lowering** the number of **features**...
     - reduces **complexity** and running time.
     - improves **generalization** capability $\Rightarrow$ model **less** prone to **overfitting**.
   - In general, a model with a **lower number of attributes** tends to **generalize better**.



2. It is possible that many attributes represent information that is **irrelevant** or of **low significance** for the problem.
   - They may separate instances that should be close in the feature space. E.g., modeling of the quality of a product based on one attribute (price) vs. two (price and date of purchase).



Two pairs of instances close by price (left) are far away when dates are taken into account (right).

One **solution** for the curse of dimensionality is to find a **projection** for the original data from a **high-dimensional** space into a **low-dimensional** one.

That can be accomplished in **two ways**:

1. **Feature extraction (feature projection)**: original features are transformed into a (smaller) set of new features.

   E.g., PCA, LDA, ICA, t-SNE, Autoencoders, ...
2. **Feature selection**: relevant features for the dataset and problem are selected, while irrelevant ones are discarded.

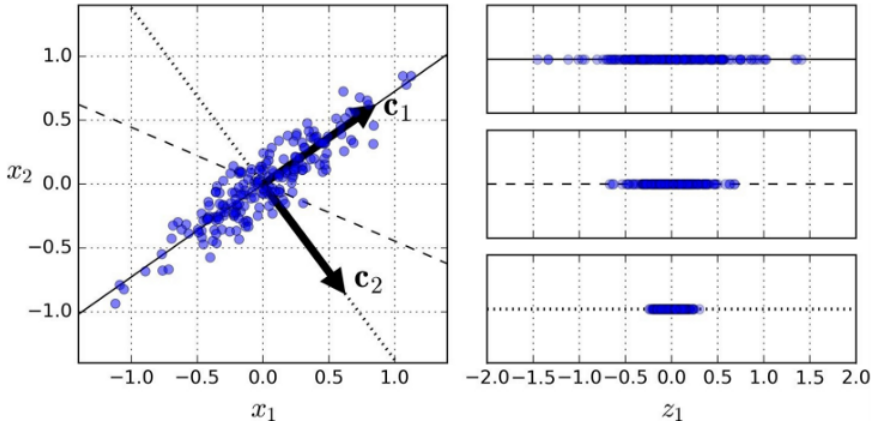
Principal Component Analysis (PCA)

PCA is a dimensionality reduction technique in which a dataset, originally in a d-**dimensional** space (d = number of features) is projected into a k-**dimensional** space of new independent features, with $k < d$.

PCA generates k **new features** for each instance from **linear combinations** of the original d ones, trying to maximize **data variance** within **each feature**.

In order to do that, the original data is projected onto a set of **orthogonal** axes (the **principal components**), selected to minimize their distances to the data points, and to maximize the variance of the data along the axes.

Example:



In the leftmost figure, 3-dimensional data is represented in two dimensions.

The continuous line and the two discontinuous ones are the three orthogonal axes (principal components) maximizing the variance of the data along them.

The three plots on the right are the projections of the original data onto those three orthogonal axes.

- **projection** = **coordinate** of the data point in the new axis = **value** of the new feature.
- The **first** principal **component** (upper right) exhibits the **biggest variance** in the proyected data.
- Variance in the **third component** (middle right) is still **significant**.
- The **second** principal **component** (lower right) shows **low variance**.
- We could build an **alternate two-dimensional representation** using the **projections** of the original data on the **first two components**.

There are several ways to find the principal components. One possibility, discussed in depth here (and, more humorously, in this video) is:

1. Build the **scatter matrix** or the **covariance matrix** of the original data.
   - The covariance matrix is a $d \times d$ matrix in which each element σ_{ij}^2 is the covariance between the values of attributes i and j.
2. Use **Eigendecomposition** or **Singular Value Decomposition (SVD)** to obtain the **eigenvectors** and **eigenvalues** from the scatter/covariance matrix.
   - **eigenvectors**: **principal components**.
   - **eigenvalues**: **variances** of the principal components.
3. **Sort** the **eigenvectors** by decreasing eigenvalues and **select** the k eigenvectors with the **largest eigenvalues** $\Rightarrow$ new k-dimensional space.
4. Build the low-dimensional data as the **projection** of the original data in the **new space**.

Using PCA makes sense when the **variance** of the data along the **principal components** is **very different** (the eigenvalues are very different).

It is necessary to standarise (**z-score**) the data before PCA, so differences in scale do not affect the choice of **principal components**.

- Attributes with bigger ranges will have bigger variances.

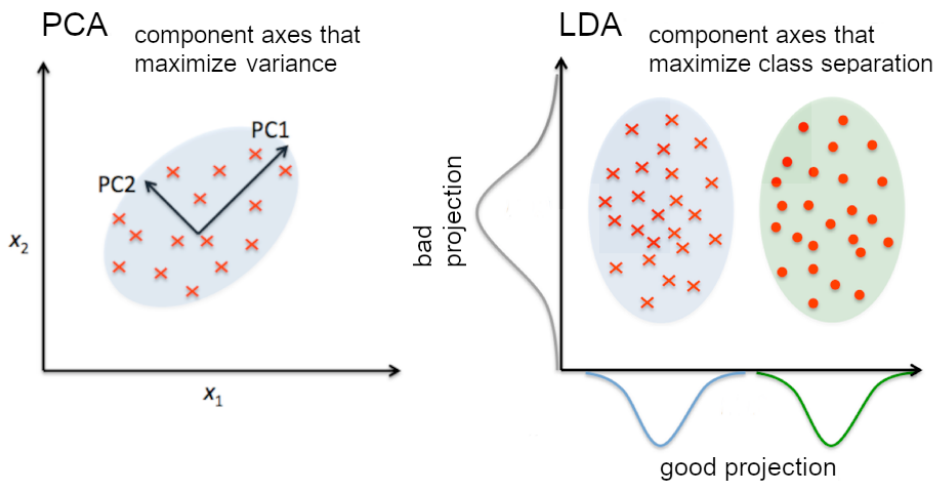
Linear Discriminant Analysis (LDA)

LDA is a technique developed for **multi-class classification** and repurposed as a dimensionality reduction algorithm.

In the latter usage, the goal of LDA is to **project** a d-dimensional dataset onto a k-dimensional space, with $k < d$, **similarly to PCA**.

The **difference** is that LDA aims for features with **good class separability** in order to avoid overfitting and reduce computational costs.

- Class data is used to select the **orthogonal axes** on which the original data will be projected, based on:
  - **Maximum separation** of examples **between classes**.
  - **Minimum separation** of examples **within each class**.
- **PCA**: **unsupervised** method.
- **LDA**: **supervised** method.



A detailed **procedure** for LDA is presented here. It is very **similar** to the one for **PCA**:

- **Main difference**: the matrix on which eigendecomposition or SVD is performed.

- In LDA is the product of **two scatter matrices**:
  - **Within-class** scatter matrix.
  - **Between-class** scatter matrix.

It might be expected that LDA outperforms PCA in multiclass classification, but it is not always so:

- **PCA outperforms** LDA if the number of **instances per class** is **low**.
- It is not uncommon to use both techniques **together**: PCA for dimensionality reduction followed by LDA.

Independent Component Analysis (ICA)

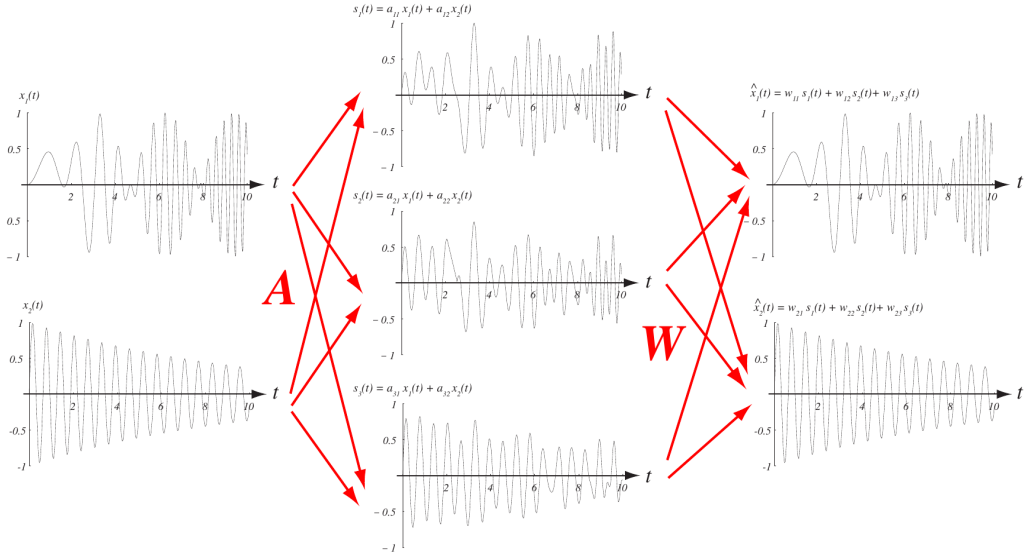
ICA is often presented as a **component extraction technique** for **Blind Source Separation** (also known as the "cocktail party problem").

Example: two people are recorded talking at the same time.

- The signals received by the microphones at any point in time will be given by **linear combinations** of the amplitudes of the two voices.
- The **coefficients** of these linear combinations will be **constant**.
- By **inferring** the values of the **coefficients**, the mixing process can be inverted $\Rightarrow$ two **clean signals** can be obtained.

More formally:

- In each time instant we have $\boldsymbol{c}$ independent **source signals** $\boldsymbol{x(t)}$.
- They are combined in $\boldsymbol{d}$ **observed signals** $\boldsymbol{s(t)}$ by a $d \times c$ matrix of **linear combinations** $\boldsymbol{A}$, such that $s(t) = A\,x(t)$, with $d \geq c$.
- The task of ICA is to find a **matrix** $\boldsymbol{W}$ such that $W = A^{-1}$ and
$$\hat{x}(t) = W s(t) = W A\, x(t)$$
where $\boldsymbol{\hat{x}(t)}$ is the **reconstruction** of the source signals.
- W can be solved by **maximum likelihood estimation**, under certain assumptions:
  - Statistically **independent** source signals.
  - Observed signals are **linear mixtures** of unobserved (source) ones.
  - At most one of the sources is **Gaussian distributed**.



Source: "Pattern Classification". Richard O. Duda, Peter E. Hart, David G. Stork. John Wiley & Sons. 2001

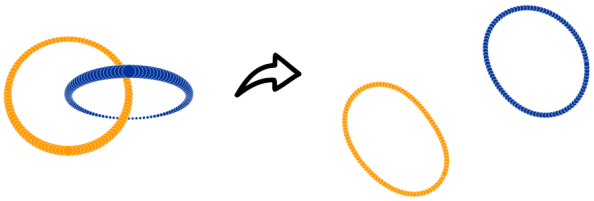
ICA can be used as a **linear dimension reduction method** ($d \geq c$), transforming the dataset into columns of independent components.

- **Observed signals**: atributes in the **high-dimensional** space.
- **Source signals**: atributes in the **low-dimenional** space

t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE is a **dimensionality reduction** technique:

- Based on **probability distributions**.
- It cannot be used on new data $\Rightarrow$ mostly used to **visualize** and **understand** high-dimensional data.
- **Non**linear technique $\Rightarrow$ works well with nonlinearly separable data.



Source: "How to use t-SNE effectively" (distill.pub)

It works by minimizing the **divergence** between two probability distributions:

1. **Pairwise similarities** of the instances in a **high-dimensional** space.
   - Pairwise similarity between x_i and x_j is the conditional probability that a point x_i chooses x_j as a neighbour if neighbours are chosen in proportion to their probability density under a Gaussian (normal distribution) centered on x_i.
2. **Pairwise similarities** of the instances in the **low-dimensional** space.
   - Initially data points (instances) distributed randomly.
   - Pairwise similarity of data points under a t-student distribution.

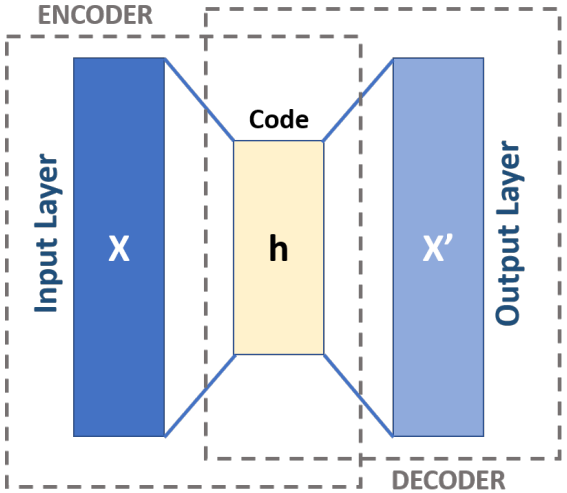
t-SNE minimises the **Kullback-Leibler** divergence between the second distribution and the first using a **gradient descent** method.

- **Data points** close in the high-dimensional space are **"pulled"** towards each other in the low-dimensional space.

Autoencoders

An autoencoder is a **neural network** trained to learn **low-dimensional encodings** of data in a higher-dimensional space.

- Usually a **MLP** with one or more hidden layers divided in two parts:
  - An **encoder** transforming the input data, or **message**, into a low-dimensional representation called **code**.
  - A **decoder** reconstructing the **message** from the **code** as perfectly as possible.
- The network is trained with the **input as the expected output**.



Source: wikipedia (Author: Michella Massi)

Autoencoders can be used as a **dimensionality reduction** technique:

- **Instances** are the **messages**.
- **Codes** are **low-dimensional representations** of the instances.
- Autoencoders can be **linear** or **nonlinear** depending on the hyperparameters of the network (activation function).